

Analyse et Exploration des données

–
Florent PRADEL

ynov
CAMPUS

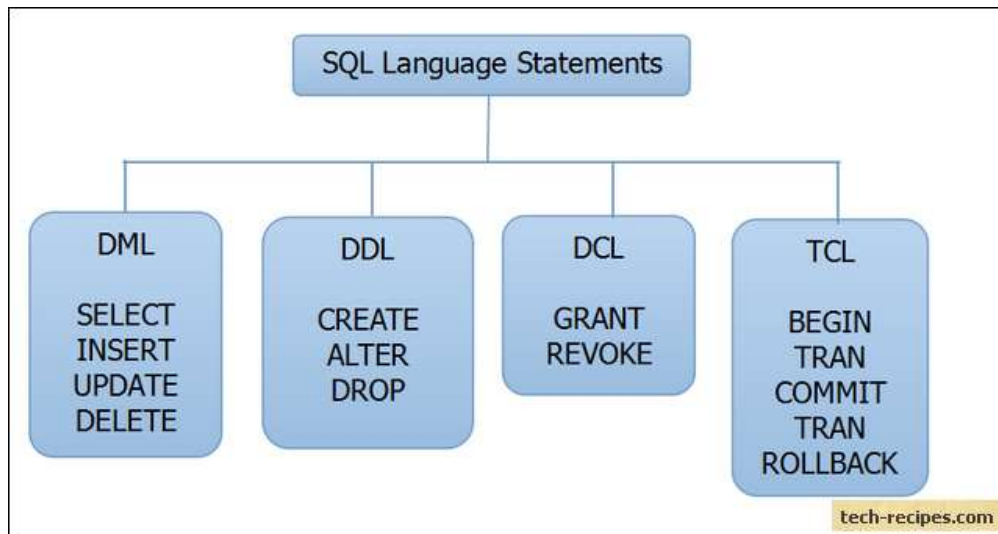
ceicom
Solutions

Les bases



Un petit rappel (j'espère) SQL

- ◎ SQL = Structured Query Language
- ◎ Langage de gestion de base de données



Les requêtes de sélection

- La sélection (SELECT) :

SELECT *[nom du champ]* **FROM** *[nom de la table];*

- Les jointures (JOIN):

SELECT [nom du champ] FROM [nom de la table 1]

JOIN *[nom de la table 2]* **ON** *[nom du champ 1] = [nom du champ 2];*

- Les conditions (WHERE):

SELECT [nom du champ] FROM [nom de la table 1]

JOIN [nom de la table 2] ON [nom du champ 1] = [nom du champ 2]

WHERE *[nom du champ 1] = « toto »;*

DML (DATA MANIPULATION LANGUAGE)

- SELECT

SELECT * FROM TABLE
WHERE condition

- INSERT

INSERT INTO table VALUES ('valeur 1', 'valeur 2', ...)

- UPDATE

UPDATE table SET nom_colonne_1 = 'nouvelle valeur'
WHERE condition

- DELETE

DELETE FROM `table`
WHERE condition

Opérateurs de comparaisons

=	Égale
<>	Pas égale
!=	Pas égale
>	Supérieur à
<	Inférieur à
>=	Supérieur ou égale à
<=	Inférieur ou égale à
IN	Liste de plusieurs valeurs possibles
BETWEEN	Valeur comprise dans un intervalle donnée (utile pour les nombres ou dates)
LIKE	Recherche en spécifiant le début, milieu ou fin d'un mot.
IS NULL	Valeur est nulle
IS NOT NULL	Valeur n'est pas nulle

Les fonctions

- SUM() calculer la somme d'un set de résultat
- MAX() obtenir le résultat maximum (fonctionne bien pour un entier)
- MIN() obtenir le résultat minimum
- COUNT() compter le nombre de lignes dans un résultat
- ROUND() arrondir la valeur
- UPPER() afficher une chaîne en majuscule
- LOWER() afficher une chaîne en minuscule
- CONCAT() concaténer des chaînes de caractères
- CURRENT_DATE() date actuelle

Exemple:

```
SELECT SUM(salaire) FROM TABLE_SALAIRE;
```

Group By

- Utilisé avec les fonctions
- Permet de grouper plusieurs résultats
- Exemple :

```
SELECT nom_joueur, id_joueur , SUM(quantité_de_sel)
FROM JOUEUR
JOIN PARTIE_FORTNITE ON id_joueur = id_joueur_fortnite
GROUP BY nom_joueur, id_joueur ;
```

Cette requête permet de calculer la somme de sel accumulé par chaque joueur sur toutes les parties de Fortnite.

Le GROUP BY contient tous les champs utilisés dans le SELECT sur lesquels il n'y a pas de fonctions appliquées.



La condition Having

- HAVING = WHERE
- Se place après un GROUP BY
- Exemple :

On veut connaître le **nombre** de transformation que possèdent chaque **personnage** uniquement pour ceux qui en ont **au moins une**.

PERSONNAGE	
Id_personnage	nom_personnage
1	Goku
2	Yamcha
3	Piccolo

TRANSFORMATION	
Id_personnage	transformation
1	Super saiyan
3	Dieu absorbé
1	Kaioken
1	Super saiyan God

Solution

```
SELECT nom_personnage, COUNT(T.transformation)
FROM PERSONNAGE P
JOIN TRANSFORMATION T ON T.id_personnage = p.id_personnage
GROUP BY nom_personnage
HAVING COUNT(T.transformation) > 0;
```

Filtre

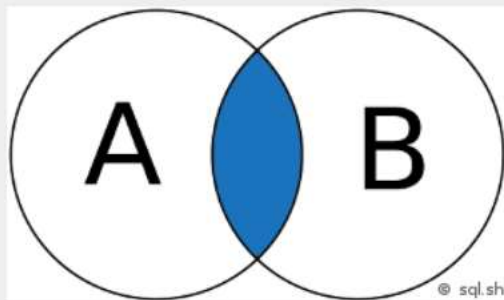


nom_personnage	COUNT(T.transformation)
Goku	3
Piccolo	1
Yamcha	0

Les jointures

11

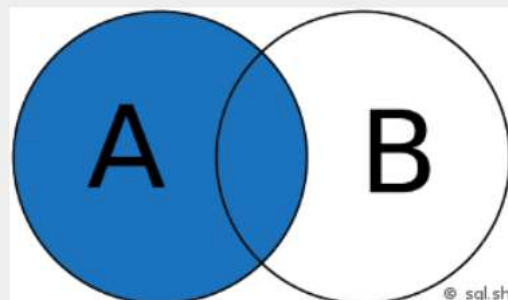
INNER JOIN



— Intersection de 2 ensembles

```
SELECT *  
FROM A  
INNER JOIN B ON A.key = B.key
```

LEFT JOIN



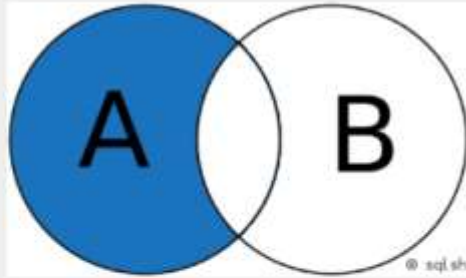
— Jointure gauche (LEFT JOIN)

```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key
```

Les jointures

12

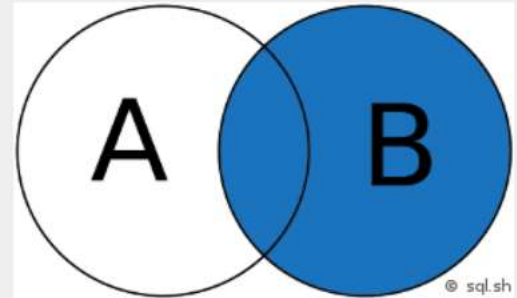
LEFT JOIN (sans l'intersection de B)



— Jointure gauche (LEFT JOIN sans l'intersection B)

```
SELECT *  
FROM A  
LEFT JOIN B ON A.key = B.key  
WHERE B.key IS NULL
```

RIGHT JOIN



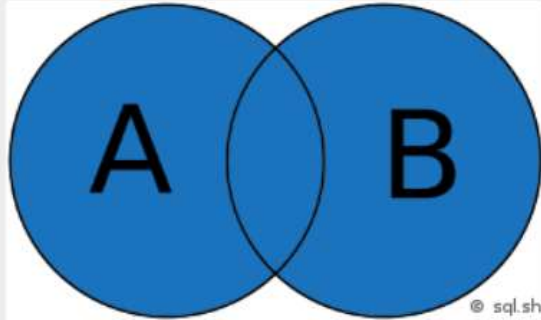
— Jointure droite (RIGHT JOIN)

```
SELECT *  
FROM A  
RIGHT JOIN B ON A.key = B.key
```

Les jointures

13

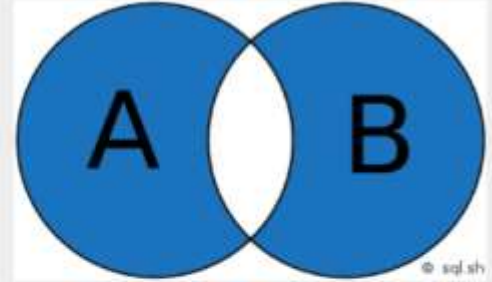
FULL JOIN



— Union de 2 ensembles

```
SELECT *  
FROM A  
FULL JOIN B ON A.key = B.key
```

FULL JOIN (sans intersection)



— Jointure pleine (FULL JOIN sans intersection)

```
SELECT *  
FROM A  
FULL JOIN B ON A.key = B.key  
WHERE A.key IS NULL  
OR B.key IS NULL
```

Les mots clés

14

- EXPLAIN (EXPLAIN PLAN sous Oracle) permet d'afficher le plan d'exécution d'une requête SQL
- **EXPLAIN** SELECT * FROM ma_table;

Résultat #1 (10x7)									
id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	CES	ALL	PRIMARY,PAR_NUMERO_COMMANDE_ET_FRS_SOCIETE...	(NULL)	(NULL)	(NULL)	135 792	Using where
1	SIMPLE	CML	eq_ref	PRIMARY	PRIMARY	1	LOCAL.CES.CODE_MODE_LIVRAISON	1	Using where
1	SIMPLE	CSF	eq_ref	PRIMARY	PRIMARY	3	LOCAL.CES.CODE_SUPPORT_FRS	1	Using where
1	SIMPLE	CLS	ref	PRIMARY,PAR_NUMERO_LIGNE_INTERNE_CDF_SOCIET...	PRIMARY	20	LOCAL.CES.CODE_FOURNISSEUR_DE_COMMANDE,LOC...	1	
1	SIMPLE	OPF	eq_ref	PRIMARY	PRIMARY	1	LOCAL.CLS.ORIGINE_PRIX_ACHAT	1	Using where
1	SIMPLE	CP	eq_ref	PRIMARY	PRIMARY	3	LOCAL.CES.CODE_PROPOSITION_FRS	1	Using where
1	SIMPLE	CTC	eq_ref	PRIMARY	PRIMARY	3	LOCAL.CES.CODE_TYPE_COMMANDE_FRS	1	Using where

- Permet de voir si la requête passe bien par les index

Les mots clés

15

- UNION permet de fusionner les résultats de 2 requêtes

```
SELECT nom, age FROM personnes
```

```
UNION
```

```
SELECT nom, age FROM autre_personnes;
```

Les résultats ne forment plus qu'un seul résultat

Les mots clés

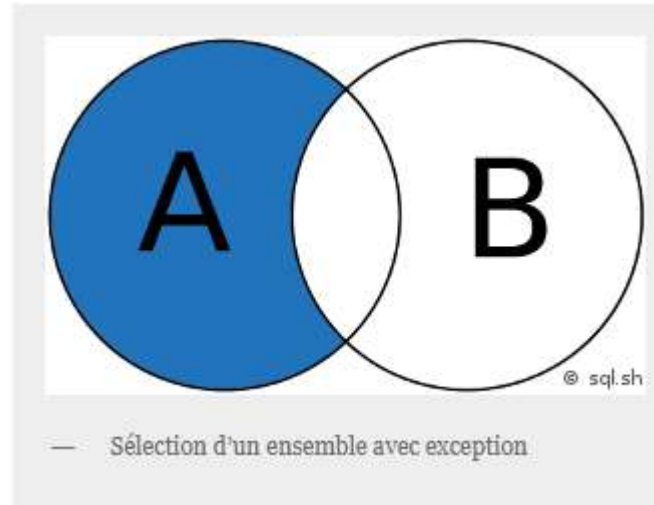
16

- MINUS permet de soustraire un résultat d'une requête à un autre

```
SELECT * FROM A
```

```
MINUS
```

```
SELECT * FROM B;
```



Les mots clés

17

- LIMIT permet de limiter le nombre de résultat d'une requête

```
SELECT * FROM comptes_bancaires  
ORDER BY solde_bancaire DESC  
LIMIT 10;
```

Retourne les 10 résultats dont le solde bancaire est le plus élevé

Les index

18

- Permet d'accéder plus rapidement aux données
- Occupe de l'espace dans la BDD
- Ralentit l'insertion de données à cause de la mise à jour de l'index

```
CREATE INDEX `nom_index` ON table (`colonne1`, `colonne2`);
```

Convention de nommage:

Préfixe “PK_” pour Primary Key (traduction : clé primaire)

Préfixe “FK_” pour Foreign Key (traduction : clé étrangère)

Préfixe “UK_” pour Unique Key (traduction : clé unique)

Préfixe “UX_” pour Unique Index (traduction : index unique)

Préfixe “IX_” pour chaque autre Index

DDL (DATA DEFINITION LANGUAGE)

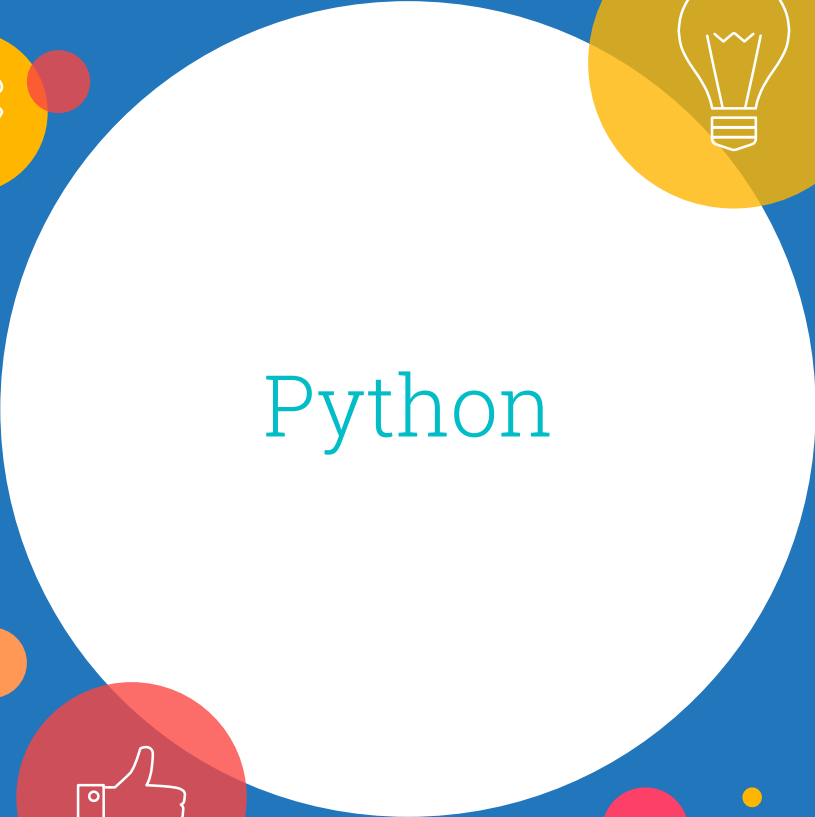
- CREATE
- ALTER
- DROP

-CREATE DATABASE nom_database;

-CREATE TABLE nom_de_la_table

(colonne1 type_donnees,
colonne2 type_donnees,
colonne3 type_donnees,
colonne4 type_donnees);

-DROP TABLE nom_de_la_table;

A large white circle containing the word "Python" in a teal, serif font. The circle is set against a solid blue background. Surrounding the circle are several decorative elements: a yellow circle with a puzzle piece icon at the top left, a yellow circle with a lightbulb icon at the top right, a red circle with a thumbs-up icon at the bottom left, and various other smaller circles in orange, pink, and teal. The word "Python" is centered within the white circle.

Python

Python

21

- Python est un langage interprété
- Plus lent que les langages compilés
- Peut utiliser des bibliothèques compilées pour plus d'efficacité
- C'est un « vrai » langage c.-à-d. types de données, branchements conditionnels,
- boucles, organisation du code en procédures et fonctions, objets et classes,
- découpage en modules.
- Très bien structuré, facile à appréhender, c'est un langage privilégié pour l'enseignement
- Mode d'exécution : transmettre à l'interpréteur Python le fichier script « .py »
- https://python.sdv.univ-paris-diderot.fr/05_boucles_comparaisons/

Python

22

- Python se base sur l'utilisation de librairie externe
- Très utilisé dans le traitement de données (Numpy, Scipy, Pandas...)
- Plus complet et libre que le langage R mais plus complexe parfois

Python

23

- Gestion des variables:
- Déclaration et initialisation en même temps:

```
>>> x = 2
```

- Typage dynamique : Python devine et adapte le type en fonction de son contenu
- Ici, x est un **entier**

```
>>> x
```

```
2
```

Python

24

- Gestion des variables:
- Une variable est sensible à la casse (var != Var != vAR)
- Vérification du type :
- Conversion de type :

```
>>> type(x)  
<class 'int'>
```

```
>>> srt(x)  
'2'
```



Python

25

- Affichage des valeurs:

```
>>> print("Hello world!")  
Hello world!
```

```
>>> var = 3  
>>> print(var)  
3
```

